CASE STUDY: UNEMPLOYMENT ANALYSIS USING EXPLORATORY DATA ANALYSIS, VISUALIZATION AND DEEP LEARNING

Assignment II — Exploratory Data Analysis and Visualization

Institution: KPR Institute of Engineering and Technology

Department: Artificial Intelligence and Data Science

Academic Year: 2024–2028

Semester: V

Submitted by: Santhiya A

Roll Number: 24AD101

Course Code: U21ADP05

Course Title: Exploratory Data Analysis and Visualization

Date of Submission: 15 October 2026

ABSTRACT

This project presents an exploratory data analysis and deep learning-based study of unemployment trends in India from 2000 to 2023. The dataset contains 24 annual observations and 17 columns, including unemployment rate, GDP growth, inflation, population, labour force participation, and other economic indicators. Data quality was assessed by checking missing values, infinite values, duplicate records, and potential outliers. Exploratory data analysis was performed using line charts, histograms, scatter plots, and correlation heatmaps to examine trends, distributions, and relationships among variables. A Multilayer Perceptron (MLP) regression model was developed to estimate unemployment rates. The simpler model achieved a test Mean Absolute Error (MAE) of 0.7277 and Root Mean Squared Error (RMSE) of 0.8484. However, the test set contained only three observations, limiting the reliability and generalizability of the results.

1. INTRODUCTION AND OBJECTIVES

1.1 Introduction

Unemployment is an important economic indicator that reflects the condition of the labour market and the availability of employment opportunities. It is influenced by several economic and demographic factors, including economic growth, inflation, labour force participation, population, and changes in economic sectors.

Exploratory Data Analysis (EDA) helps identify patterns, distributions, trends, and relationships within a dataset through statistical techniques and visualizations. Deep learning techniques can also be applied to estimate numerical outcomes using multiple input variables.

This project analyses India's unemployment rate from 2000 to 2023 using data obtained from the World Bank Indicators API. The dataset contains 24 annual observations and 17 columns. Python libraries are used for data processing, statistical analysis, visualization, and the development of a Multilayer Perceptron (MLP) regression model.

The study focuses on understanding unemployment trends, examining relationships between economic indicators, and evaluating the model's ability to estimate unemployment rates. Since the dataset contains a limited number of annual observations, the findings and model performance must be interpreted cautiously.

1.2 Objectives

The main objectives of this project are:

1. To collect and analyse real-world unemployment data and related economic indicators from the World Bank.
2. To examine data quality by checking missing values, duplicate records, infinite values, and potential outliers.
3. To perform univariate, bivariate, and multivariate exploratory data analysis.
4. To create at least five meaningful visualizations and interpret the patterns observed.
5. To preprocess the data and divide it chronologically into training, validation, and testing sets.
6. To develop and evaluate a deep learning MLP regression model for estimating unemployment rates.
7. To compare model configurations using appropriate regression evaluation metrics.
8. To identify the limitations of the analysis and suggest possible improvements for future work.

2. DATASET DESCRIPTION

2.1 Data Source

The dataset used in this project was collected from the World Bank Indicators API, which provides economic, demographic, and social indicators for countries worldwide. India was selected as the study country, and annual observations from 2000 to 2023 were used to analyse unemployment and its relationship with other economic indicators.

The unemployment rate is treated as the target variable, while the remaining numerical indicators are used as input features for exploratory analysis and deep learning.

2.2 Dataset Overview

The final dataset contains 24 observations and 17 columns, including the year, unemployment rate, and 15 additional economic and demographic indicators.

Property	**Description**
Data source	World Bank Indicators API
Country	India
Time period	2000–2023
Number of observations	24
Number of columns	17
Target variable	unemployment_rate
Data type	Annual numerical time-series data
Application	Unemployment trend analysis and regression

2.3 Description of Dataset Fields

The dataset contains the following fields:

S. No.	**Field name**	**Description**
1	year	Year of observation
2	unemployment_rate	Unemployment rate
3	gdp_growth	Annual GDP growth rate
4	gdp_per_capita	GDP per capita
5	inflation	Inflation rate
6	labor_force_participation	Labour force participation rate
7	population	Total population
8	urban_population_percent	Percentage of population living in urban areas

9	internet_users_percent	Percentage of individuals using the internet
10	foreign_direct_investment	Foreign direct investment indicator
11	trade_percent_gdp	Trade as a percentage of GDP
12	life_expectancy	Life expectancy at birth
13	female_labor_participation	Female labour force participation rate
14	agriculture_value_added	Agriculture value added indicator
15	industry_value_added	Industry value added indicator
16	services_value_added	Services value added indicator
17	gross_capital_formation	Gross capital formation indicator

2.4 Dataset Characteristics

The dataset is numerical and time-dependent, with one observation for each year in the selected period. It contains economic, demographic, labour-market, and infrastructure-related variables. The unemployment rate is used as the prediction target for the MLP regression model.

Because the dataset contains only 24 observations, statistical patterns and predictive results must be interpreted cautiously. A larger dataset covering additional countries or more frequent observations could support more robust analysis.

2.5 Descriptive Statistical Analysis

Descriptive statistical analysis was performed to understand the central tendency, variability, and range of the numerical variables. The analysis included count, mean, standard deviation, minimum, maximum, and quartile values.

The dataset contains 24 observations for each variable. The unemployment rate has a mean of 7.272 and a standard deviation of 0.925. Its minimum value is 4.172, while its maximum value is 7.859. GDP growth has a mean of 6.164 and a standard deviation of 3.094, with values ranging from −5.778 to 9.690.

Internet usage shows considerable variation, with a mean of 16.147% and a maximum of 60.252%. The mean labour force participation rate is 55.444%, while female labour force participation has a mean of 29.870%. These descriptive statistics provide an initial understanding of the dataset and support the selection of suitable visualizations for further exploratory analysis.

The statistics describe the observed data and do not, by themselves, establish causal relationships between economic indicators and unemployment.

The descriptive analysis further shows that gross capital formation has a mean of 34.146 and a standard deviation of 4.256, with values ranging from 25.677 to 41.951. GDP per capita has a mean of 1,352.983, while population has a mean of approximately 1.263 billion. Life expectancy has a mean of 67.453 years. The wide differences in measurement scales across variables indicate that standardization is appropriate before training the MLP model. Overall, the statistical summary provides

a foundation for examining individual variable distributions and relationships between unemployment and other indicators.

3. EXPLORATORY DATA ANALYSIS AND PREPROCESSING

3.1 Data Quality Assessment

The dataset was examined to assess its quality before performing exploratory analysis and deep learning. It contains 24 observations and 17 columns covering the period from 2000 to 2023.

The missing-value check identified no missing values in any column. No infinite values, duplicate rows, or duplicate years were found. Therefore, no missing-value imputation or duplicate removal was required for the current dataset.

3.2 Outlier Detection

Potential outliers were identified using the Interquartile Range (IQR) method. The IQR was calculated as the difference between the third quartile (Q3) and the first quartile (Q1). Values below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR were flagged for inspection.

The analysis flagged observations in variables including unemployment rate, GDP growth, inflation, internet usage, and agriculture value added. These observations were retained because an IQR flag alone does not prove that a value is erroneous. In time-series economic data, unusual observations may represent genuine changes in economic conditions.

3.3 Feature Selection and Transformation

The year column was used to preserve the chronological order of observations and was excluded from the model's input features. The unemployment rate was selected as the target variable, while the remaining 15 numerical indicators were used as input features.

Numerical input features were standardized using StandardScaler. The scaler was fitted on the training data and then applied to the validation and test data to avoid using information from those sets when estimating the scaling parameters.

3.4 Chronological Data Splitting

The dataset was divided chronologically into training, validation, and test sets rather than randomly shuffled. This approach preserves the temporal order of the observations.

Dataset	Years	Number of observations
Training set	2000–2017	18
Validation set	2018–2020	3
Test set	2021–2023	3
Total	2000–2023	24

The training set was used to fit the model, the validation set was used to monitor model performance during training, and the test set was used for final evaluation.

3.5 Preprocessing Limitations

The dataset contains only 24 annual observations, with three observations each in the validation and test sets. Consequently, model evaluation metrics may vary considerably with individual observations. The results should be interpreted as preliminary rather than as evidence of reliable forecasting performance.

4: Data Visualization and Interpretation

4.1 Unemployment Rate Trend in India (2000–2023)

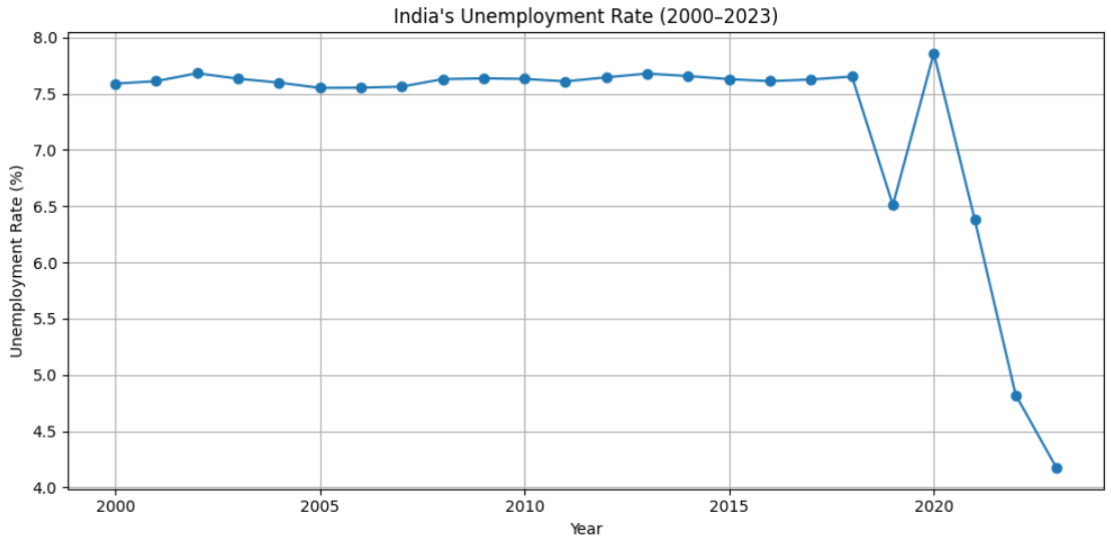


Figure 4.1: Trend of Unemployment Rate in India (2000–2023)

What the Visualization Shows

This line chart illustrates the annual changes in India's unemployment rate from 2000 to 2023. The year is represented on the X-axis, while the unemployment rate is represented on the Y-axis.

Why This Visualization Was Created

A line chart was selected to examine changes in the unemployment rate over time and identify overall trends, fluctuations, and noticeable changes during the study period.

Insights Obtained

The unemployment rate remained around 7.5%–7.7% for much of the earlier period. In the later years, a noticeable decline occurred, with the rate reaching approximately 4.17% in 2023. This indicates a change in the recorded unemployment trend during the final years of the dataset. However, the chart alone cannot establish the causes of this change.

4.2 — GDP Growth Trend in India (2000–2023)



Figure 4.2: GDP Growth Trend in India (2000–2023)

What the Visualization Shows

This line chart illustrates the annual changes in India's GDP growth rate from 2000 to 2023. The X-axis represents the year, and the Y-axis represents the GDP growth rate.

Why This Visualization Was Created

A line chart was selected to identify fluctuations in economic growth, observe long-term trends, and examine unusual changes during the study period.

Insights Obtained

India's GDP growth rate fluctuated throughout the period. A notable decline occurred in 2020, when GDP growth reached approximately −5.78%. Growth rebounded in 2021. These changes highlight the variation in India's economic performance over time. However, the chart alone cannot establish the causes of these fluctuations.

4.3 Distribution of Unemployment Rate in India (2000–2023)

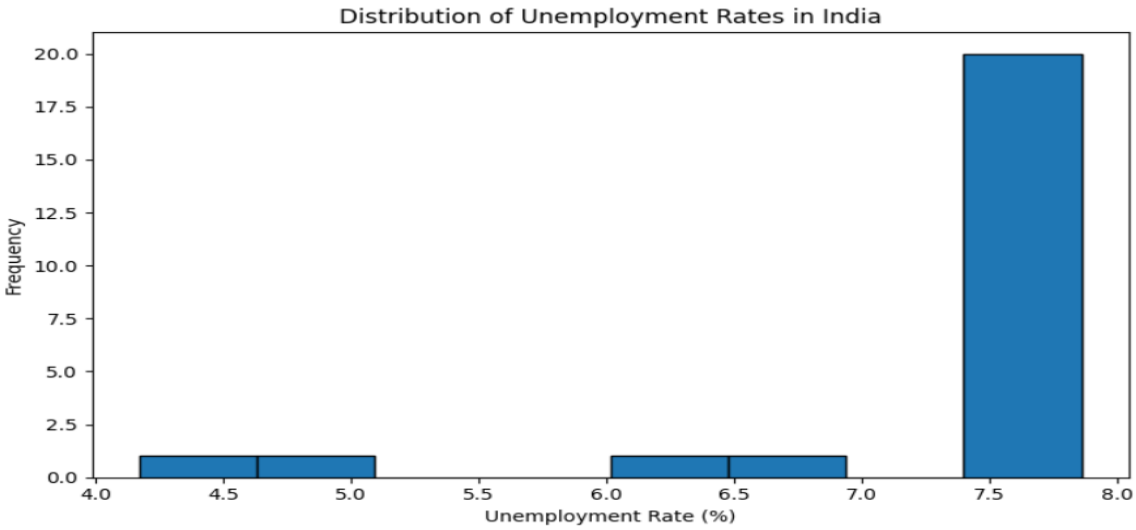


Figure 4.3: Distribution of Unemployment Rate in India (2000–2023)

What the Visualization Shows

This histogram displays the distribution of annual unemployment rate observations in India from 2000 to 2023. The X-axis represents the unemployment rate intervals, while the Y-axis represents the frequency of observations in each interval.

Why This Visualization Was Created

A histogram was selected to understand the distribution of unemployment rate values, identify the intervals where observations are concentrated, and observe whether the distribution contains noticeable variation or extreme values.

Insights Obtained

Most observations from the earlier years are concentrated around unemployment rates of approximately 7.4%–7.9%. The lower values recorded in the later years extend the distribution towards the lower end. This indicates that the unemployment rate values are not uniformly distributed across the study period. Since the dataset contains only 24 annual observations, the histogram should be interpreted cautiously.

4.4 Correlation Analysis of Economic Indicators

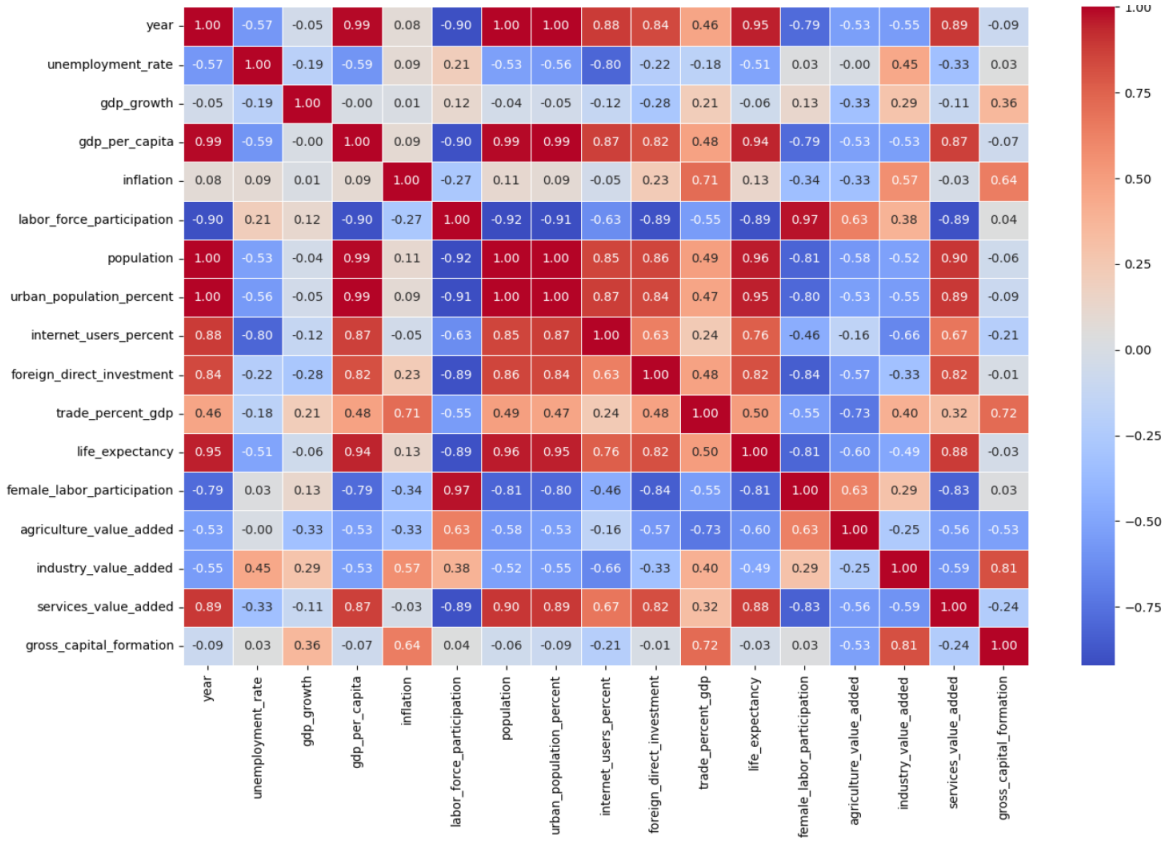


Figure 4.4: Correlation Heatmap of Economic Indicators in India (2000–2023)

What the Visualization Shows

The correlation heatmap presents the pairwise correlation coefficients between the numerical variables in the dataset. The values range from −1 to +1. Positive values indicate that two variables tend to move in the same direction, while negative values indicate that they tend to move in opposite directions. Values closer to zero indicate a weaker linear relationship.

Why This Visualization Was Created

The heatmap was created to identify relationships among economic indicators, explore potential associations with the unemployment rate, and understand how the variables relate to one another before developing the deep learning model.

Insights Obtained

The heatmap indicates a weak negative linear correlation of approximately −0.19 between GDP growth and the unemployment rate. It also shows strong positive correlations between GDP per capita and population, and between labour force participation and female labour force participation. These relationships describe associations in this dataset; they do not establish causation. The results should also be interpreted cautiously because the dataset contains only 24 annual observations.

4.5 Relationship Between GDP Growth and Unemployment Rate

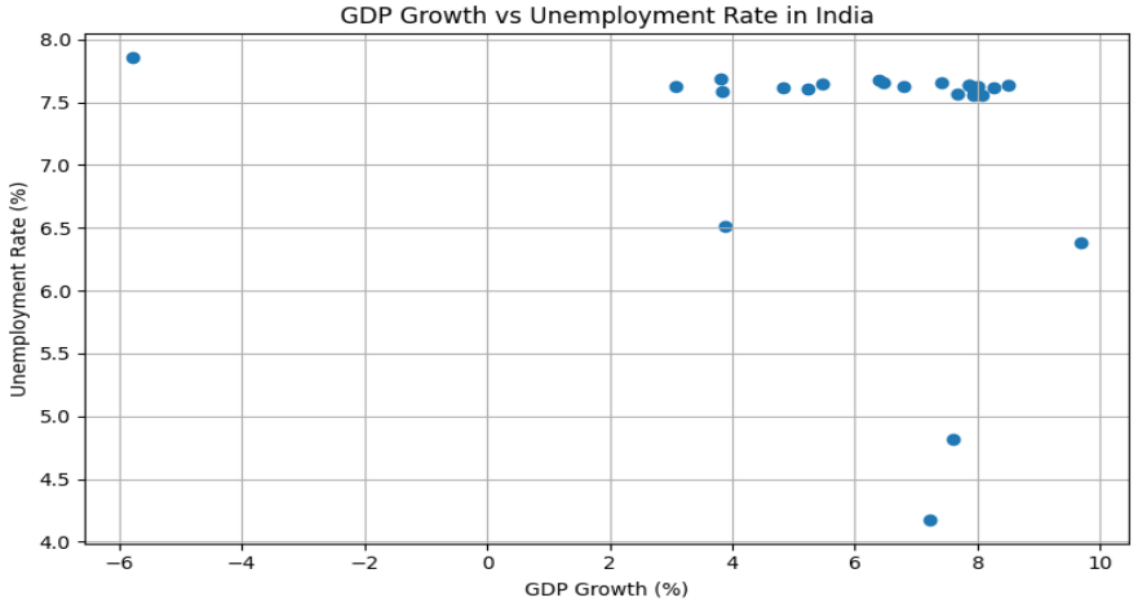


Figure 4.5: Relationship Between GDP Growth and Unemployment Rate in India (2000–2023)

What the Visualization Shows

This scatter plot illustrates the relationship between GDP growth and the unemployment rate. GDP growth is represented on the X-axis, and the unemployment rate is represented on the Y-axis. Each point represents one year's observation.

Why This Visualization Was Created

A scatter plot was selected to examine whether changes in GDP growth are associated with changes in the unemployment rate and to identify any visible patterns or unusual observations.

Insights Obtained

Most observations are concentrated around GDP growth rates of approximately 3%–8% and unemployment rates of around 7.5%–7.7%. The plot does not show a clear linear relationship between the two variables, which is consistent with the weak negative correlation of approximately −0.19. However, the relationship may be influenced by other economic factors, and the small number of observations limits the conclusions that can be drawn.

4.6 Outlier Analysis of Unemployment Rate



Figure 4.6: Box Plot of Unemployment Rate in India (2000–2023)

What the Visualization Shows

The box plot summarizes the distribution of unemployment rate observations using the median, lower quartile, upper quartile, and potential outliers. Points beyond the whiskers may be identified as potential outliers according to the box plot's statistical rule.

Why This Visualization Was Created

The box plot was used to examine the spread and central tendency of unemployment rates and identify observations that differ from the main distribution.

Insights Obtained

The unemployment rate observations are concentrated around the median of approximately 7.62, while the later years contain several noticeably lower values. The IQR-based analysis identified five potential outliers in the unemployment rate variable. These observations were retained because a statistical outlier is not necessarily a data error; unusual values may represent genuine changes in the recorded indicator. The findings should be interpreted cautiously because the dataset contains only 24 annual observations.

4.7 Probability Distribution of Unemployment Rate

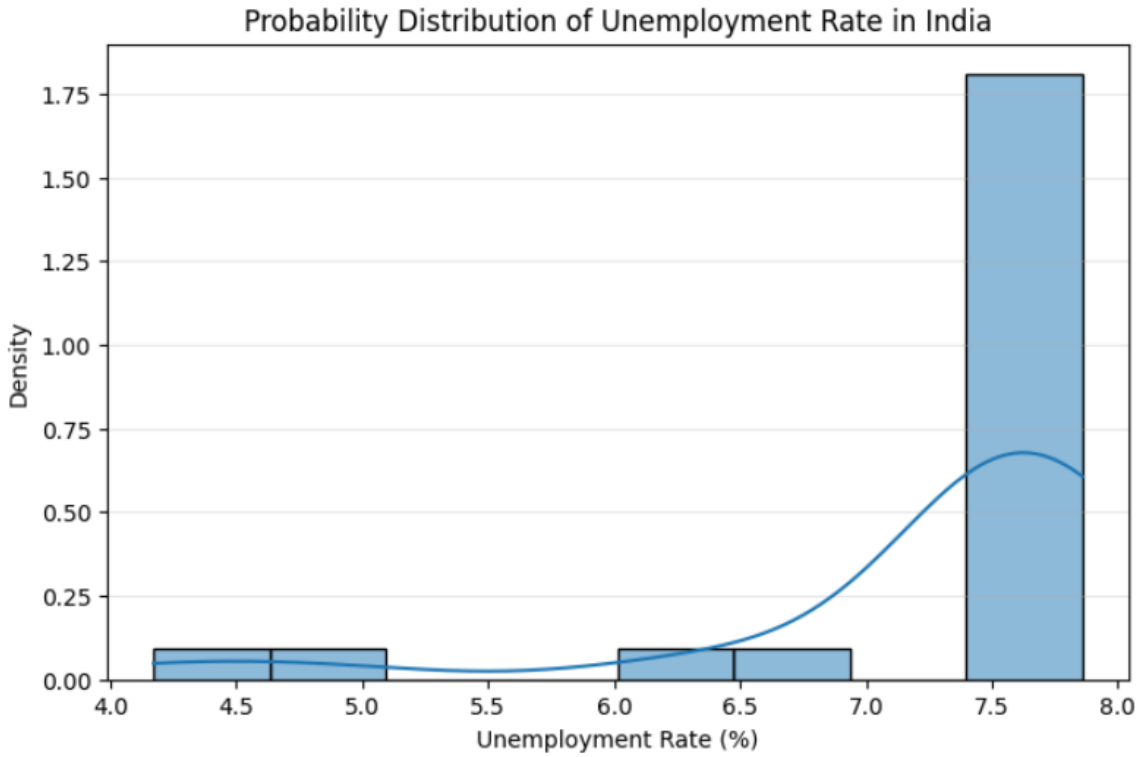


Figure 4.7: Estimated Distribution of Unemployment Rate in India (2000–2023)

What the Visualization Shows

This visualization combines a histogram and a Kernel Density Estimation (KDE) curve to represent the distribution of annual unemployment rate observations. The histogram shows the density across intervals, while the KDE curve provides a smoothed estimate of the distribution.

Why This Visualization Was Created

The visualization was created to examine the concentration and spread of unemployment rate values and to explore the general shape of their distribution.

Insights Obtained

The observations are concentrated around the higher unemployment rate values recorded during much of the earlier period, while lower values from recent years extend the distribution towards the lower end. The estimated curve helps illustrate this variation. However, the dataset contains only 24 annual observations, so the estimated distribution should not be treated as a reliable description of all possible unemployment rates.

5: Deep Learning Model Development

5.1 Model Selection and Architecture

A Multilayer Perceptron (MLP) neural network was selected to estimate India's unemployment rate using the available economic indicators. An MLP is a feedforward neural network that learns relationships between input features and a numerical target through interconnected layers of neurons.

The problem was formulated as a regression task because the target variable, unemployment rate, is continuous. The model uses economic indicators such as GDP growth, inflation, population, labour force participation, and other numerical features as inputs. The year and unemployment rate columns were excluded from the input features.

A simpler MLP architecture was selected as the final model after comparing its test performance with that of the original, more complex MLP. The simpler architecture contains an input layer, one hidden dense layer with eight neurons and ReLU activation, a dropout layer, and an output layer with one neuron and linear activation.

Component	Configuration
Input layer	15 numerical features
Hidden dense layer	8 neurons
Hidden activation	ReLU
Dropout layer	0.3
Output layer	1 neuron
Output activation	Linear
Task	Regression

5.2 Data Splitting and Feature Scaling

The dataset was divided into training, validation, and testing subsets based on chronological order. This approach preserves the time sequence of the annual observations and allows the model to be evaluated on years that occur after the training period.

The training set contains observations from 2000 to 2017 and was used to learn the model parameters. The validation set contains observations from 2018 to 2020 and was used to monitor model performance during training. The test set contains observations from 2021 to 2023 and was reserved for final evaluation.

StandardScaler was used to standardize the input features. The scaler was fitted only on the training data and then applied to the validation and test data. This prevents information from the validation and test sets from influencing the scaling parameters learned during training.

Dataset	Year range	Number of observations
Training set	2000–2017	18
Validation set	2018–2020	3
Test set	2021–2023	3
Total	2000–2023	24

Standardization Formula

Standardization transforms a feature by subtracting its training-set mean and dividing by its training-set standard deviation.

$$z = \frac{x - \mu}{\sigma}$$

$\mu =$ Mean
$\sigma =$ Standard Deviation

5.3 Model Training and Hyperparameter Configuration

The selected MLP regression model was implemented using TensorFlow and Keras. The model was trained to minimize Mean Squared Error (MSE), which measures the average squared difference between actual and predicted unemployment rates.

The Adam optimizer was used to update the model parameters during training. ReLU activation was applied in the hidden layer, while a linear activation was used in the output layer to predict continuous values. L2 regularization and dropout were included to help control overfitting.

Early stopping was applied by monitoring the validation loss. Training was stopped when the validation loss did not improve for 20 consecutive epochs, and the model weights from the best validation-loss epoch were restored. The maximum number of training epochs was set to 200, with a batch size of four observations.

5.3 Model Training and Hyperparameter Configuration

Hyperparameter table

Hyperparameter	Configuration
Optimizer	Adam
Learning rate	0.001
Loss function	Mean Squared Error (MSE)
Evaluation metric during training	Mean Absolute Error (MAE)
Hidden layer neurons	8
Hidden activation	ReLU
Dropout rate	0.3
L2 regularization coefficient	0.05
Maximum epochs	200
Batch size	4
Early stopping patience	20 epochs
Random seed	42

Model comparison

Model	Test MAE	Test RMSE	Test R^2
Original MLP	1.3808	1.4344	−1.3971
Simpler MLP	0.7277	0.8484	0.1615

5.4 Model Evaluation and Results

The performance of the final MLP regression model was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). These metrics compare the predicted unemployment rates with the actual values in the test set.

MAE measures the average absolute difference between actual and predicted values. RMSE measures prediction errors while assigning greater weight to larger errors. R^2 indicates how well the predictions explain the variation in the observed target values.

The simpler MLP model achieved a test MAE of 0.7277, an RMSE of 0.8484, and an R^2 of 0.1615. It performed better than the original MLP on all three reported test metrics. However, the test set contains only three observations, so the results are preliminary and do not establish that the model will perform similarly on unseen data.

5.5 Training and Validation Loss

5.5.1 Training Loss vs. Validation Loss



Figure 5.1: Training and Validation Loss of the MLP Regression Model

What the Visualization Shows

This graph displays the training loss and validation loss across the training epochs. The X-axis represents the number of epochs, while the Y-axis represents the Mean Squared Error (MSE) loss.

Why This Visualization Was Created

The loss curves were plotted to monitor the model's learning process, compare performance on training and validation data, and identify possible signs of overfitting.

Insights Obtained

The loss curves show how the model's prediction error changes during training. The training process used early stopping to monitor validation loss and restore the model weights from the best validation-loss epoch. Comparing the two curves helps assess whether improvements on the training set are accompanied by improvements on the validation set. The small number of validation observations limits the reliability of this assessment.

5.5.2 — Actual vs. Predicted Unemployment Rate

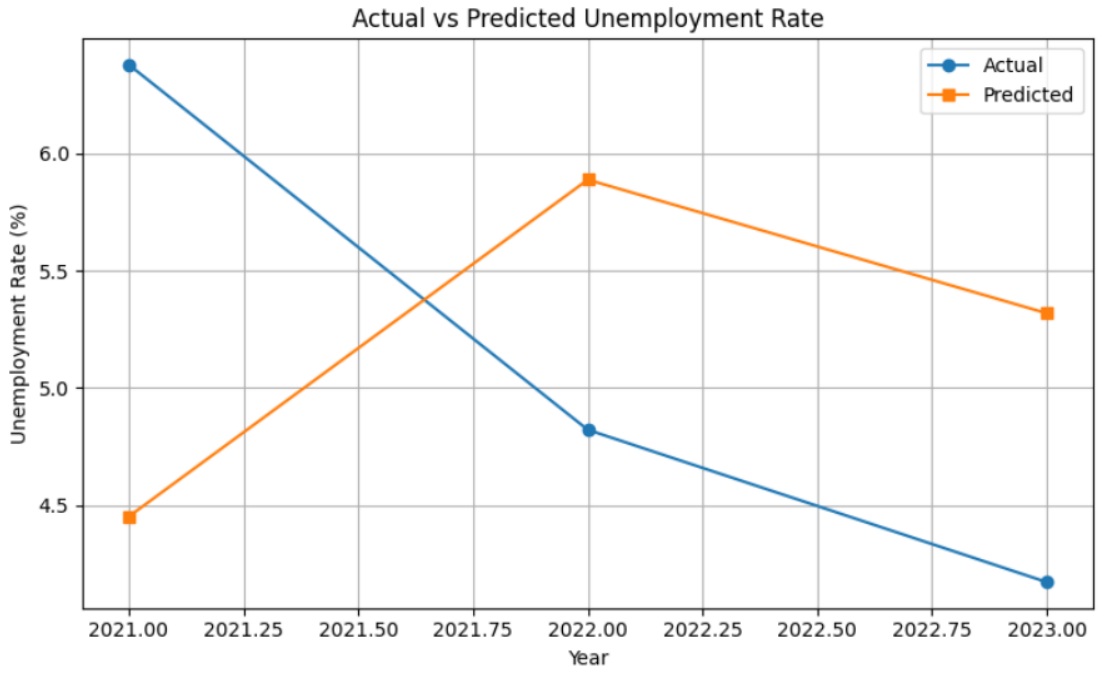


Figure 5.2: Actual vs. Predicted Unemployment Rate Using the MLP Model

What the Visualization Shows

This chart compares the actual unemployment rates with the values predicted by the final MLP regression model for the test period from 2021 to 2023. It helps visualize the differences between observed and predicted values.

Why This Visualization Was Created

The chart was created to assess how closely the model's predictions match the actual unemployment rates and to identify years with larger prediction errors.

Insights Obtained

The model did not predict all three test observations accurately. The test MAE was 0.7277 and the test RMSE was 0.8484. These results indicate that prediction errors remain. Since the test set contains only three annual observations, the comparison is insufficient to establish reliable forecasting performance.

5.5.3 Residual Error Distribution

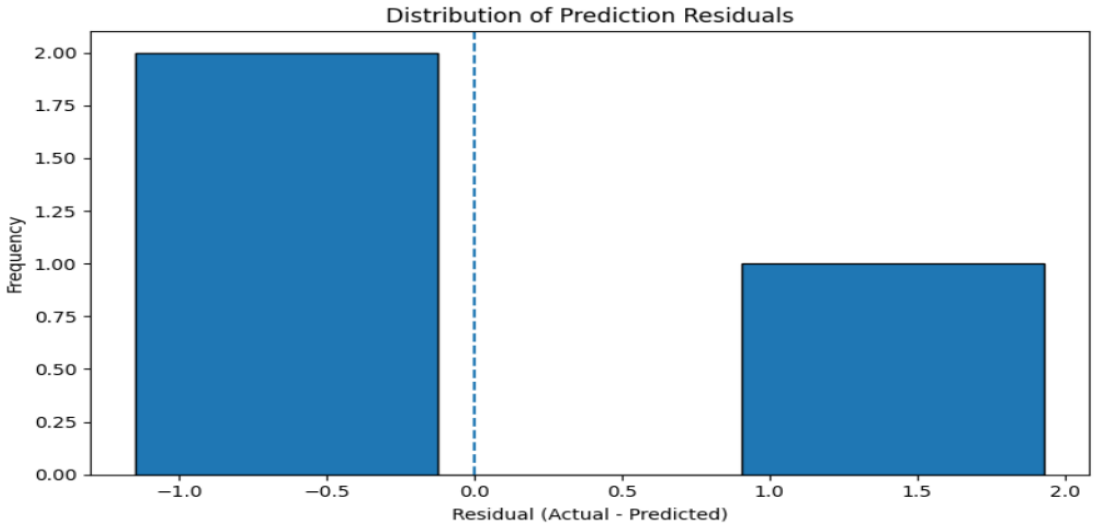


Figure 5.3: Distribution of Residual Errors in the MLP Regression Model

What the Visualization Shows

This histogram displays the distribution of residual errors for the model's test predictions. A residual is calculated by subtracting the predicted value from the actual value.

Why This Visualization Was Created

The histogram was created to examine the magnitude and distribution of prediction errors and identify whether the model tends to overestimate or underestimate the unemployment rate.

Insights Obtained

The histogram summarizes the errors made by the model on the test observations. Residuals close to zero indicate predictions close to the actual values, while larger absolute residuals indicate larger prediction errors. Because the test set contains only three observations, the histogram has very limited information and should not be used to draw conclusions about the overall error distribution.

5.5.4 Residuals vs. Predicted Values

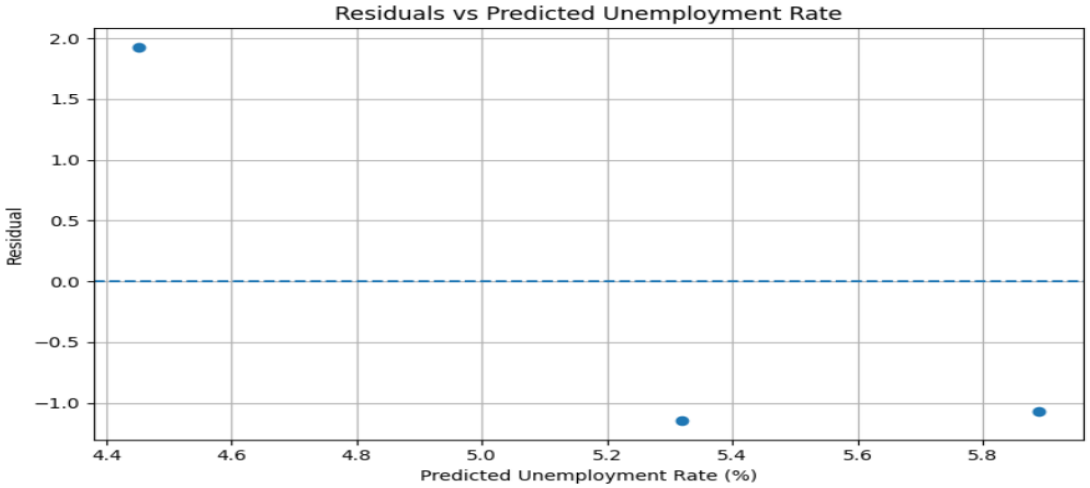


Figure 5.4: Residuals vs. Predicted Values for the MLP Regression Model

What the Visualization Shows

This scatter plot displays the residual errors against the unemployment rates predicted by the MLP model. The X-axis represents predicted values, and the Y-axis represents residual errors, calculated as actual values minus predicted values.

Why This Visualization Was Created

The plot was created to examine whether prediction errors show any visible patterns across the predicted values and to identify observations with relatively large errors.

Insights Obtained

The plot helps identify whether residuals are distributed around zero or show a noticeable pattern. A pattern may suggest that the model does not capture some aspects of the relationship between the input features and the target variable. However, the test set contains only three observations, so it is not sufficient to establish whether the residuals are randomly distributed or whether the model assumptions are satisfied.

6. Feature Importance Analysis

6.1 Permutation Feature Importance

Permutation feature importance was used to investigate how individual input features affected the predictive performance of the final MLP regression model. In this method, the values of one feature are shuffled while the other features remain unchanged. The resulting change in model error is used to estimate the feature's importance.

A larger positive increase in Mean Absolute Error (MAE) indicates that shuffling the feature worsened the model's predictions. A negative importance value indicates that the model's error decreased after shuffling that feature. Negative values do not necessarily mean that a feature is unimportant; they can occur because of sampling variability and interactions between features.

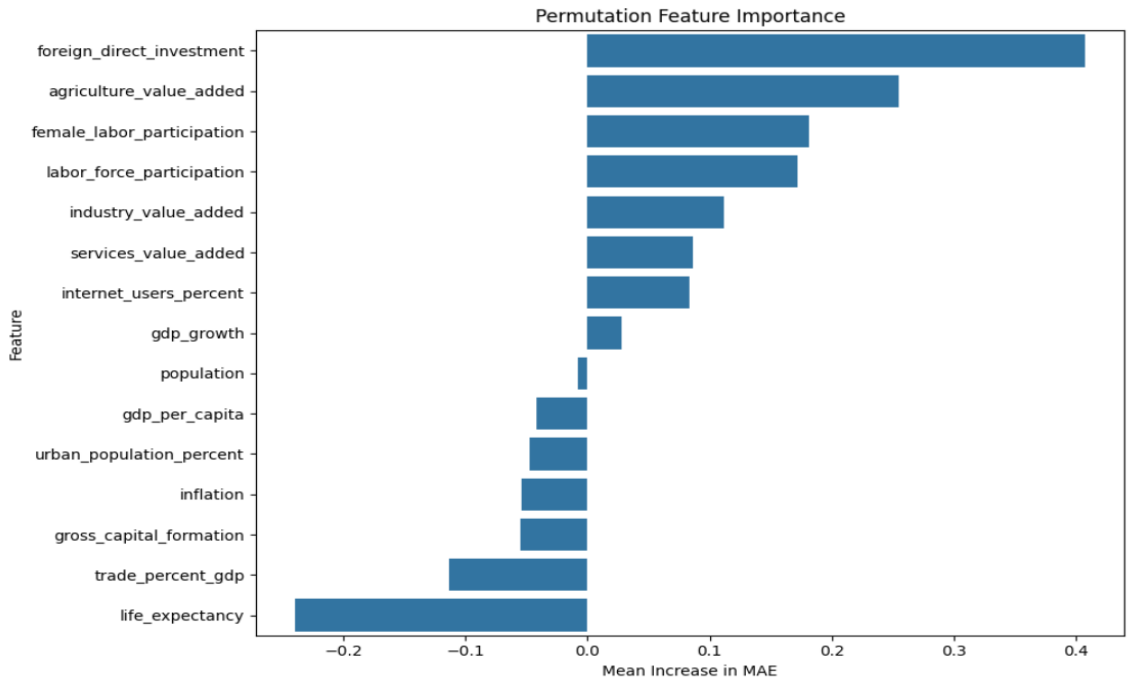


Figure 6.1: Permutation Feature Importance of Economic Indicators for the MLP Model

6.2 Interpretation of Feature Importance

The permutation importance results showed that foreign direct investment had the largest positive importance value, followed by agriculture value added, female labour force participation, and labour force participation. These features produced larger increases in test MAE when their values were shuffled.

However, some features had negative importance values, meaning that shuffling them reduced the measured test MAE in this evaluation. This may reflect the instability of the model's predictions on a very small test set rather than a genuine lack of predictive value.

The test set contains only three observations. Therefore, these feature importance results are preliminary and should not be interpreted as proof of causal relationships or as reliable rankings of the most important economic factors affecting unemployment.

7. Conclusion and Future Scope

7.1 Conclusion

This project presented an exploratory data analysis and deep learning-based study of unemployment trends in India from 2000 to 2023. The dataset contained 24 annual observations and 17 columns representing unemployment and other economic indicators.

Data quality checks identified no missing values, infinite values, or duplicate records in the dataset used for analysis. Line charts, histograms, a correlation heatmap, scatter plots, and a box plot were used to explore trends, distributions, relationships, and potential outliers. The analysis showed a noticeable decline in the recorded unemployment rate in the later years, while GDP growth fluctuated and experienced a significant decline in 2020.

An MLP regression model was developed to estimate unemployment rates. The simpler MLP achieved a test MAE of 0.7277, RMSE of 0.8484, and R^2 of 0.1615, outperforming the original MLP on the reported test metrics. However, the test set contained only three observations. Therefore, the results are preliminary and do not establish that the model can reliably predict future unemployment rates.

7.2 Challenges

The main challenge was the limited number of annual observations available for model development. The small validation and test sets made it difficult to evaluate generalization reliably. The dataset also contains economic indicators that may be related to one another, making feature importance results sensitive to the available observations and model behaviour.

7.3 Future Scope

Future work could extend the dataset with more observations, additional relevant economic indicators, and more recent verified data. The analysis could be improved by comparing the MLP model with suitable statistical and machine learning baselines, using more robust time-series validation methods, and systematically tuning model hyperparameters.

Further research could also examine the relationships between unemployment and economic indicators across different demographic groups and sectors. These improvements would help assess whether the observed patterns and model predictions remain consistent across different periods.

8. Literature Review

Exploratory Data Analysis (EDA) is an important step in understanding datasets before building predictive models. It involves examining data distributions, identifying unusual observations, and investigating relationships between variables using statistical summaries and visualizations. Histograms, box plots, scatter plots, and correlation heatmaps help reveal patterns that may not be apparent from raw data alone.

The World Bank's World Development Indicators provide economic and social statistics that can be used to study development trends and relationships between indicators. In this project, World Bank data were used to examine unemployment trends in India alongside other economic variables.

A Multilayer Perceptron (MLP) is a feedforward neural network that can learn nonlinear relationships between input features and a target variable. However, neural networks require careful model evaluation, particularly when the dataset is small. Standardization, regularization, and validation-based early stopping can help control the training process, but they cannot eliminate the limitations caused by having very few observations.

Based on these concepts, this project combines EDA, visualization, and an MLP regression model to investigate unemployment trends in India. The findings are exploratory, and the limited number of annual observations restricts the reliability of the model evaluation.

9. References

1. World Bank. (n.d.). *Unemployment, total (% of total labor force) (national estimate) – India*. World Development Indicators. https://data.worldbank.org/indicator/SL.UEM.TOTL.NE.ZS?locations=IN
2. Wickham, H. (2014). Tidy data. *Journal of Statistical Software, 59*(10), 1–23. https://doi.org/10.18637/jss.v059.i10
3. scikit-learn developers. (n.d.). *StandardScaler*. scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
4. TensorFlow. (n.d.). *tf.keras.callbacks.EarlyStopping*. TensorFlow API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping

APPENDIX A: SOURCE CODE

The source code for the Unemployment Analysis project was developed using Python in Google Colab. The implementation covers data collection, data preprocessing, exploratory data analysis, visualization, deep learning model development, model evaluation, and feature importance analysis.

The complete executable notebook is named Unemployment_Analysis_EDA.ipynb and is available in the project's GitHub repository.

A.1 Importing Libraries

This section contains the Python libraries used for data processing, visualization, preprocessing, and deep learning model development.

```
import requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

print("All libraries imported successfully!")
```

A.2 Data Collection

This section contains the code used to collect India's annual unemployment and economic indicator data from the World Bank API for 2000–2023.

```
url = (
    "https://api.worldbank.org/v2/country/IND/"
    "indicator/SL.UEM.TOTL.ZS"
)

params = {
    "date": "2000:2023",
    "format": "json",
    "per_page": 100
}

response = requests.get(url, params=params)
response.raise_for_status()

data = response.json()[1]

df = pd.DataFrame([
    {
        "country": row["country"]["value"],
        "year": int(row["date"]),
        "unemployment_rate": row["value"]
    }
    for row in data
])

print(df.head())
print("Dataset shape:", df.shape)
```

A.3 Data Cleaning and Preprocessing

This section contains the code for examining missing values, infinite values, duplicate records, data types, and potential outliers.

```
print("Dataset dimensions:", df.shape)

print("\nAll column names:")
print(df.columns.tolist())

print("\nMissing values in each column:")
print(df.isnull().sum())

print("\nStatistical summary:")
display(df.describe())
```

```
# Check missing values again
print("Missing values before cleaning:")
print(df.isnull().sum())

print("\nTotal missing values:", df.isnull().sum().sum())

# Replace infinite values with missing values
import numpy as np

df = df.replace([np.inf, -np.inf], np.nan)

# Fill missing values in numeric columns using column medians
for column in df.columns:
    if column != "year":
        df[column] = df[column].fillna(df[column].median())

print("\nMissing values after cleaning:")
print(df.isnull().sum())

print("\nFinal dataset shape:", df.shape)
display(df.head())
```

```
print("Missing values:", df.isnull().sum().sum())

print("\nCheck 2004 row:")
display(df[df["year"] == 2004].T)

print("\nData types:")
print(df.dtypes)

print("\nDuplicate rows:", df.duplicated().sum())
```

A.4 Exploratory Data Analysis and Visualization

This section contains the code used to generate the following visualizations:

1. Unemployment rate trend.
2. GDP growth trend.
3. Unemployment rate histogram.
4. Correlation heatmap.
5. GDP growth versus unemployment scatter plot.
6. Unemployment rate box plot.
7. Estimated unemployment rate distribution using a histogram and KDE curve.

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.plot(
    df["year"],
    df["unemployment_rate"],
    marker="o"
)

plt.title("India's Unemployment Rate (2000-2023)")
plt.xlabel("Year")
plt.ylabel("Unemployment Rate (%)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```python
plt.figure(figsize=(10, 5))

plt.plot(
    df["year"],
    df["gdp_growth"],
    marker="o"
)

plt.title("India's GDP Growth (2000-2023)")
plt.xlabel("Year")
plt.ylabel("GDP Growth (%)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```python
plt.figure(figsize=(8, 5))

plt.hist(
    df["unemployment_rate"],
    bins=8,
    edgecolor="black"
)

plt.title("Distribution of Unemployment Rates in India")
plt.xlabel("Unemployment Rate (%)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

```python
import seaborn as sns
import matplotlib.pyplot as plt

correlation = df.corr(numeric_only=True)

plt.figure(figsize=(14, 10))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)

plt.title("Correlation Heatmap of Economic Indicators")
plt.tight_layout()
plt.show()
```

```python
plt.figure(figsize=(8, 5))

plt.scatter(
    df["gdp_growth"],
    df["unemployment_rate"]
)

plt.title("GDP Growth vs Unemployment Rate in India")
plt.xlabel("GDP Growth (%)")
plt.ylabel("Unemployment Rate (%)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 5))

sns.boxplot(y=df["unemployment_rate"])

plt.title("Box Plot of Unemployment Rate in India (2000-2023)")
plt.ylabel("Unemployment Rate (%)")
plt.grid(axis="y", alpha=0.3)
plt.show()
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 5))

sns.histplot(
    data=df,
    x="unemployment_rate",
    bins=8,
    kde=True,
    stat="density"
)

plt.title("Probability Distribution of Unemployment Rate in India")
plt.xlabel("Unemployment Rate (%)")
plt.ylabel("Density")
plt.grid(axis="y", alpha=0.3)
plt.show()
```

A.4 Descriptive Statistical Analysis

This section contains the code used to calculate descriptive statistics, including count, mean, standard deviation, minimum, quartiles, and maximum values.

```python
# Final data-quality check

print("Dataset shape:", df.shape)

print("\nMissing values in each column:")
print(df.isnull().sum())

print("\nInfinite values in each numeric column:")
print(
    np.isinf(df.select_dtypes(include="number"))
    .sum()
)

print("\nDuplicate rows:", df.duplicated().sum())
print("Duplicate years:", df["year"].duplicated().sum())

print("\nYear range:", df["year"].min(), "-", df["year"].max())
```

A.6 Data Splitting and Feature Scaling

This section contains the code used to split the dataset chronologically into training, validation, and test sets and standardize the input features using StandardScaler.

```
train_mask = df["year"] <= 2017
val_mask = df["year"].between(2018, 2020)
test_mask = df["year"] >= 2021

X_train = X.loc[train_mask]
y_train = y.loc[train_mask]

X_val = X.loc[val_mask]
y_val = y.loc[val_mask]

X_test = X.loc[test_mask]
y_test = y.loc[test_mask]

print("Training rows:", len(X_train))
print("Validation rows:", len(X_val))
print("Testing rows:", len(X_test))
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

print("Training data standardized successfully!")
print("Training shape:", X_train_scaled.shape)
print("Validation shape:", X_val_scaled.shape)
print("Testing shape:", X_test_scaled.shape)
```

A.7 MLP Regression Model

This section contains the code used to define, compile, and train the Multilayer Perceptron regression model, including its architecture, optimizer, loss function, regularization, dropout, and early stopping configuration.

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input
from tensorflow.keras.callbacks import EarlyStopping

# Make results more reproducible
tf.keras.utils.set_random_seed(42)

# Build the MLP regression model
model = Sequential([
    Input(shape=(X_train_scaled.shape[1],)),
    Dense(32, activation="relu"),
    Dropout(0.2),
    Dense(16, activation="relu"),
    Dense(1, activation="linear")
])

# Compile the model
model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

# Display model architecture
model.summary()
```

A.8 Model Evaluation

This section contains the code used to compare the original and simpler MLP models using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np
import pandas as pd

# Predict unemployment rates for test data
y_pred = model.predict(X_test_scaled).flatten()

# Calculate evaluation metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("TEST RESULTS")
print("MAE:", round(mae, 4))
print("RMSE:", round(rmse, 4))
print("R-squared:", round(r2, 4))

# Compare actual and predicted values
results = pd.DataFrame({
    "Year": df.loc[X_test.index, "year"].values,
    "Actual Unemployment Rate": y_test.values,
    "Predicted Unemployment Rate": y_pred
})

display(results)
```

A.9 Model Evaluation Visualizations

This section contains the code used to generate the training and validation loss curves, actual-versus-predicted plot, residual error histogram, and residuals-versus-predicted plot.

```
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Mean Squared Error")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 5))

plt.plot(
    results["Year"],
    results["Actual Unemployment Rate"],
    marker="o",
    label="Actual"
)

plt.plot(
    results["Year"],
    results["Predicted Unemployment Rate"],
    marker="s",
    label="Predicted"
)

plt.title("Actual vs Predicted Unemployment Rate")
plt.xlabel("Year")
plt.ylabel("Unemployment Rate (%)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
results["Residual"] = (
    results["Actual Unemployment Rate"]
    - results["Predicted Unemployment Rate"]
)

plt.figure(figsize=(8, 5))

plt.hist(
    results["Residual"],
    bins=3,
    edgecolor="black"
)

plt.axvline(0, linestyle="--")

plt.title("Distribution of Prediction Residuals")
plt.xlabel("Residual (Actual - Predicted)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 5))

plt.scatter(
    results["Predicted Unemployment Rate"],
    results["Residual"]
)

plt.axhline(0, linestyle="--")

plt.title("Residuals vs Predicted Unemployment Rate")
plt.xlabel("Predicted Unemployment Rate (%)")
plt.ylabel("Residual")
plt.grid(True)
plt.tight_layout()
plt.show()
```

A.10 Permutation Feature Importance

This section contains the code used to calculate and visualize permutation feature importance and examine changes in test-set MAE when individual features are shuffled.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Sort using the correct, existing DataFrame 'importance_df'
sorted_importance_df = importance_df.sort_values(
    "Mean MAE Increase", ascending=False
)

plt.figure(figsize=(10, 7))
sns.barplot(
    data=sorted_importance_df,
    x="Mean MAE Increase",
    y="Feature"
)
plt.title("Permutation Feature Importance")
plt.xlabel("Mean Increase in MAE")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```